



The Internet of Things: Connecting Our World

Session(2): IoT Stack

The Communication Challenge

Imagine sending a simple "Hello" across the world – from Cairo to California. How does that message, made of mere electrical signals, successfully travel continents and arrive intact, understood, and quickly? This seemingly effortless act faces several complex challenges:

- How to break messages into small, manageable packets?
- How to deliver these packets efficiently across continents and oceans?
- How to ensure data integrity and prevent corruption during transmission?
- How to convert the raw numerical data back into recognizable text like "Hello" upon arrival?



Without Standards = Chaos

When different companies, such as HP, Cisco, IBM, and Microsoft, operate independently without common rules, a state of chaos ensues:

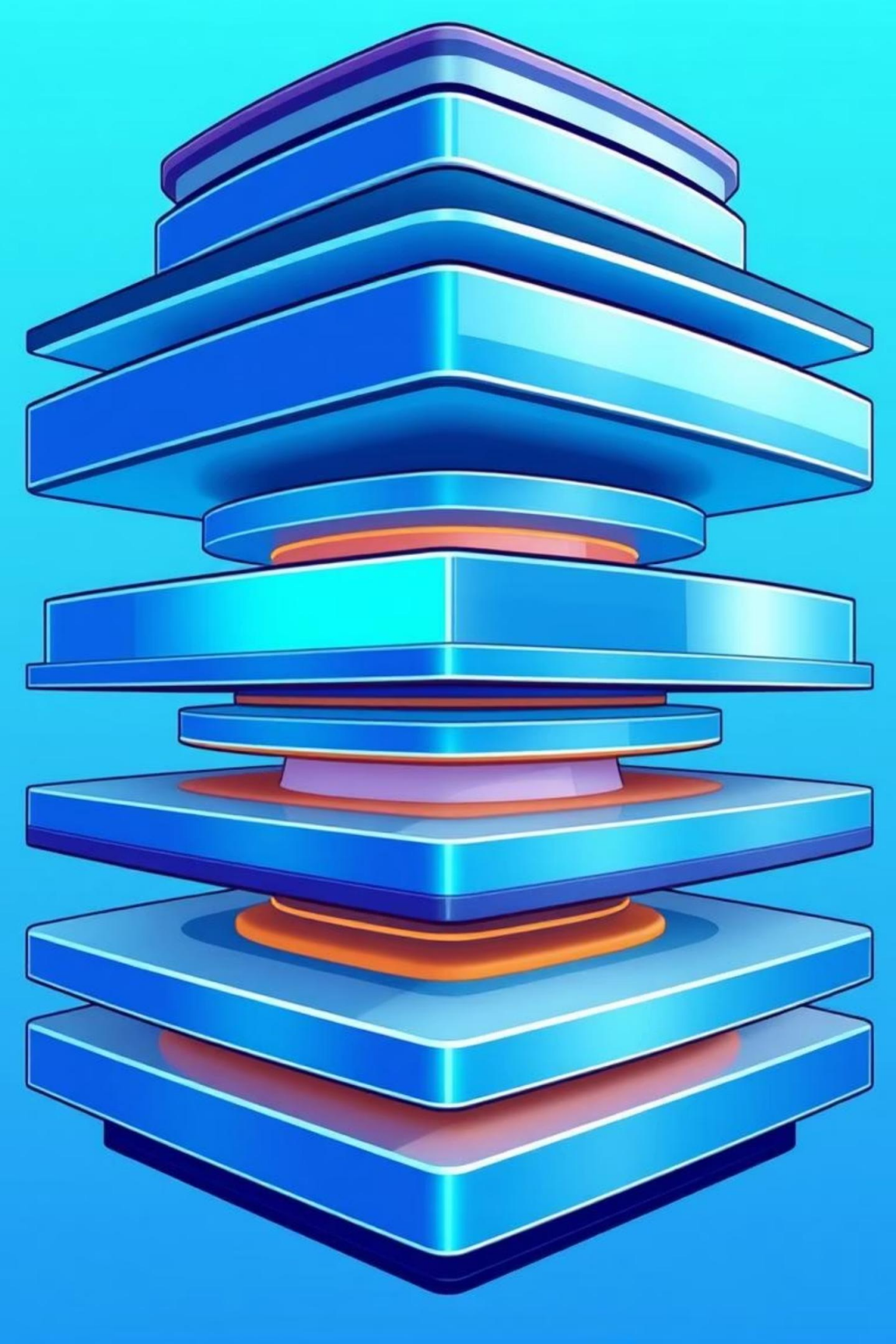
- Each company creates their own protocols
- Devices can't understand each other
- Communication becomes impossible
- No interoperability between systems



The Solution: The Stack Model

The "stack model" provides an elegant and effective solution to the complex communication challenges we've discussed:

- Create a unified model with layers
- Each layer has a clear, specific function
- Layers work like a team - each specializes in their area
- Pass work to the next layer without interfering



Why Layers Are Important



Division & Simplification

Break complex problems into clear steps



Specialization

Each layer focuses on one specific task



Flexibility

Change one layer without affecting the whole system



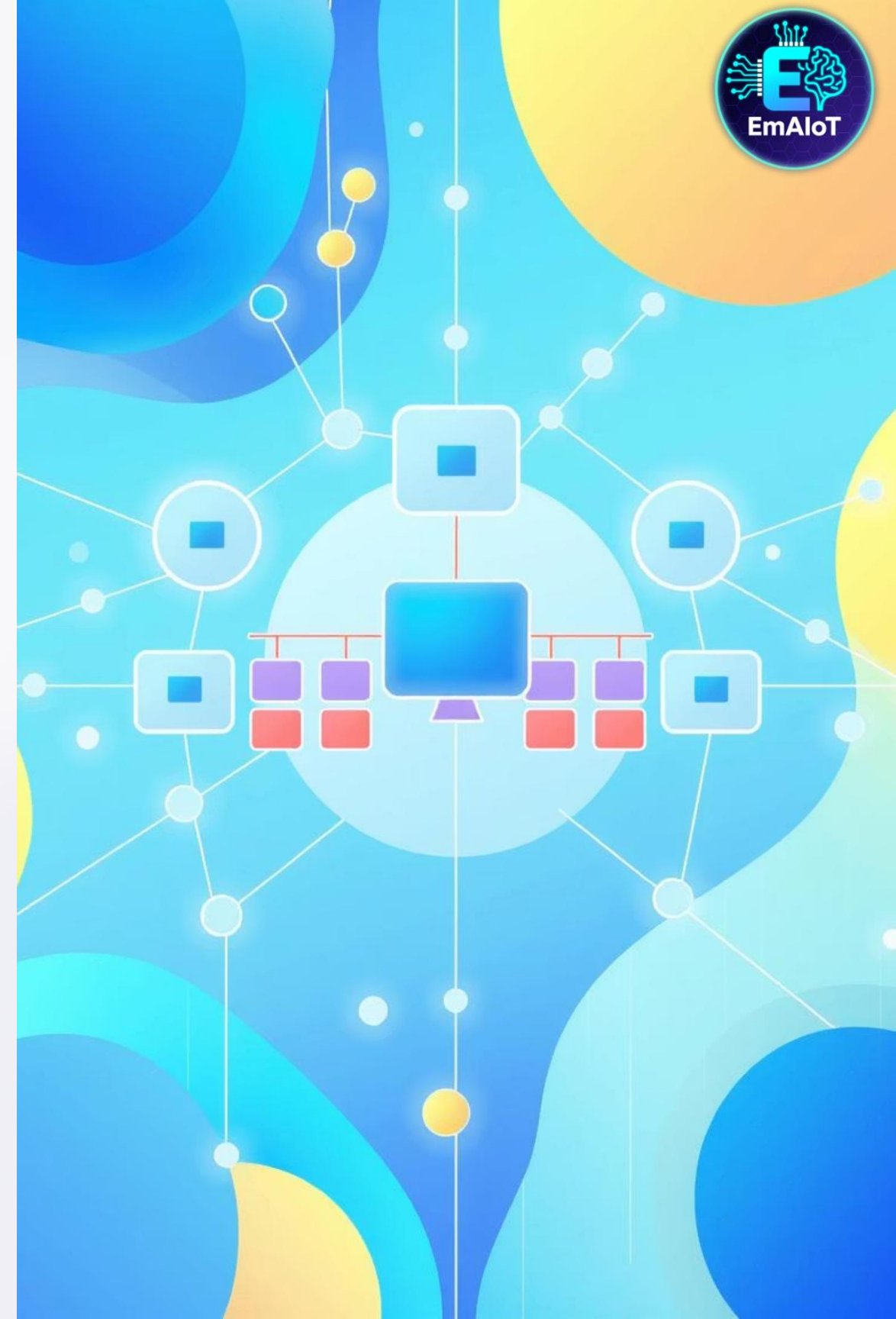
Universal Standards

Any device can communicate with any other device



Scalability

Develop new protocols without breaking existing ones



Real-World Example: Evolution Without Breaking

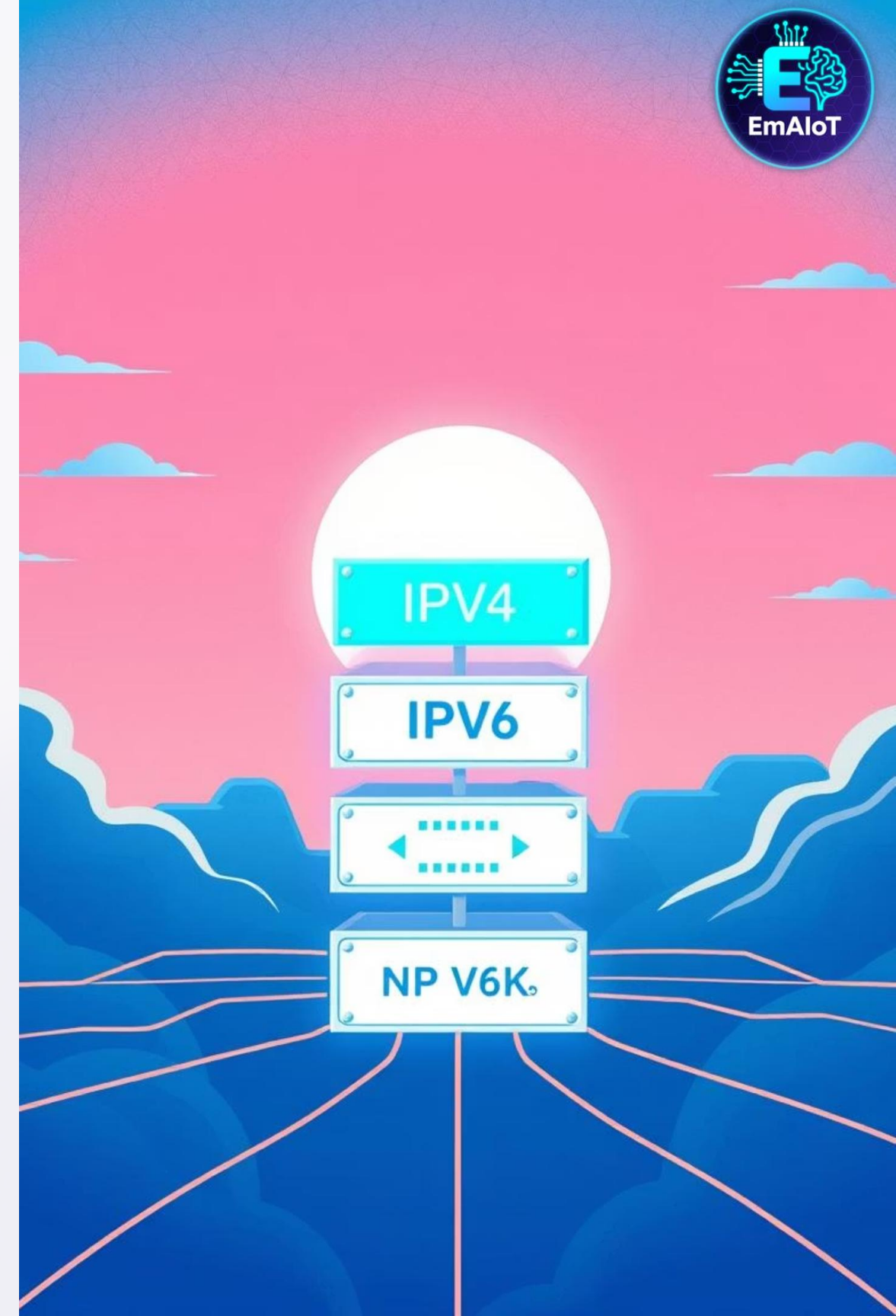
Example: IPv4 to IPv6 transition

Only the Network layer changed

All other layers remained the same

Applications still work normally

Transport, Physical layers unaffected

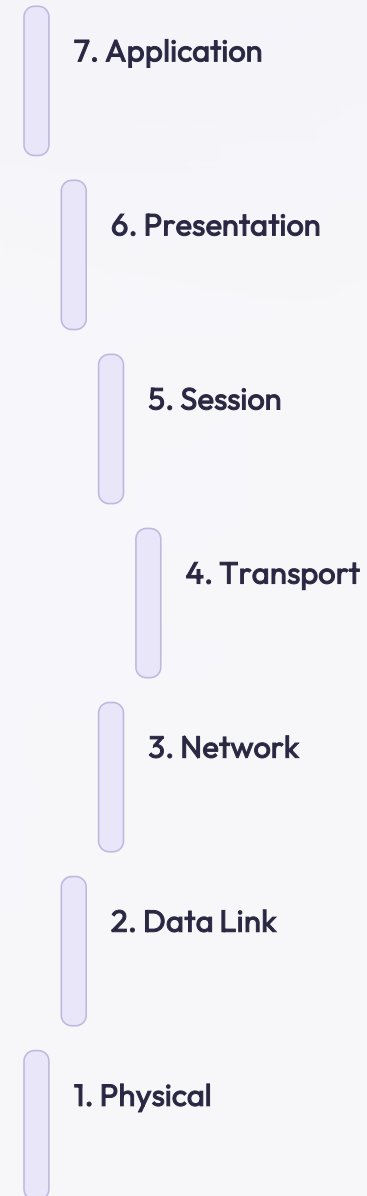


The OSI Model: 7-Layer Framework

The OSI model, which stands for **Open Systems Interconnection**, is a conceptual framework that standardizes the functions of a telecommunication or computing system into seven distinct layers.

It was created by the **ISO (International Organization for Standardization)** to provide a universal reference model for networking, allowing diverse communication systems to interoperate.

Each of the 7 layers has specific responsibilities, working together to ensure seamless data transmission and reception:



OSI Model Layers Explained

1

Application Layer

User interface (HTTP, FTP, SMTP)

2

Presentation Layer

Data formatting, encryption, compression

3

Session Layer

Manages connections between applications

4

Transport Layer

Reliable data transfer (TCP, UDP)

5

Network Layer

Routing and addressing (IP)

6

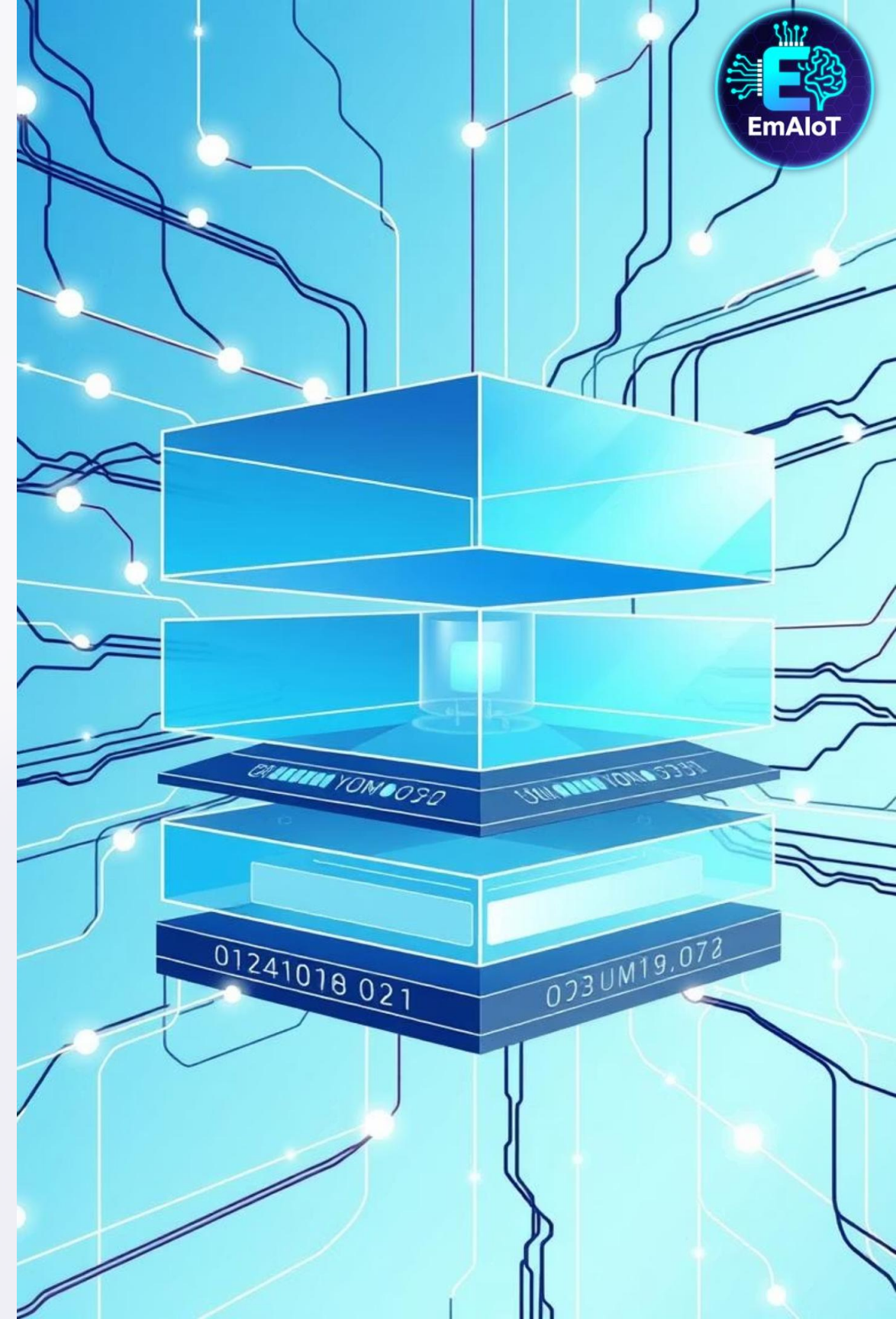
Data Link Layer

Frame formatting, error detection

7

Physical Layer

Physical transmission (cables, signals)



TCP/IP Model: The Internet's Foundation

The TCP/IP Model, which stands for **Transmission Control Protocol/Internet Protocol**, is the foundational model that the internet actually uses. Unlike the more theoretical OSI model, TCP/IP focuses on practical implementation and how data is exchanged across networks.

It provides a robust framework that enables seamless communication across diverse devices and networks, serving as the blueprint for real-world internet operations. This model is generally considered more practical than the OSI model because it directly reflects the architecture of the internet.

The TCP/IP model simplifies the complex process of network communication into four distinct layers, each with specific responsibilities:

01

Application Layer

02

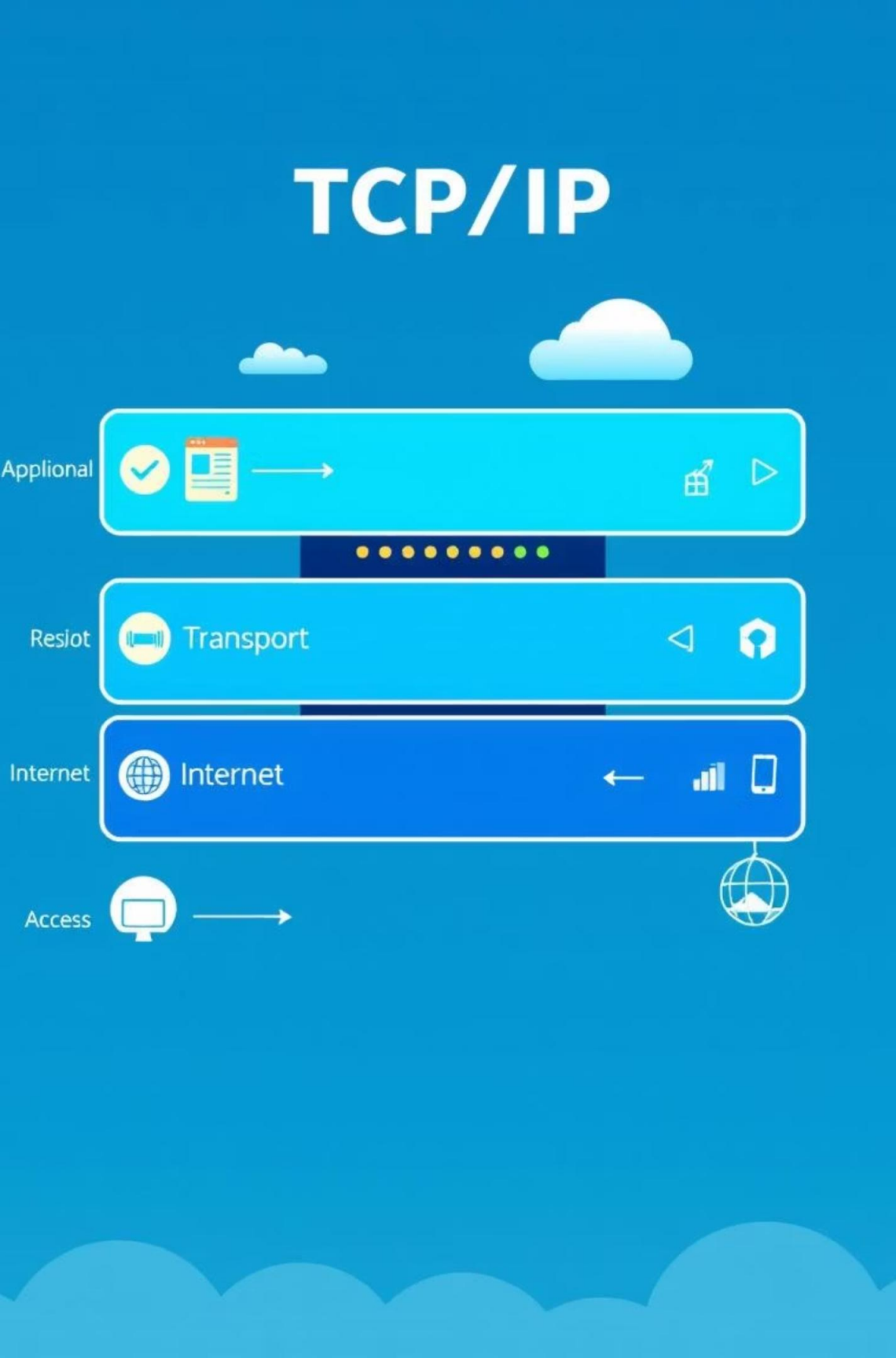
Transport Layer

03

Internet Layer

04

Network Access Layer



TCP/IP Model Layers Explained

01

Application Layer

User applications and services (HTTP, FTP, SMTP, DNS)

02

Transport Layer

End-to-end communication (TCP, UDP)

03

Internet Layer

Packet routing across networks (IP, ICMP)

04

Network Access Layer

Physical network interface (Ethernet, WiFi)



TCP vs UDP: Two Transport Protocols

When data travels across a network, different transport protocols dictate how that data is packaged and delivered. TCP and UDP are two fundamental protocols, each with distinct characteristics and use cases.

TCP (Transmission Control Protocol)

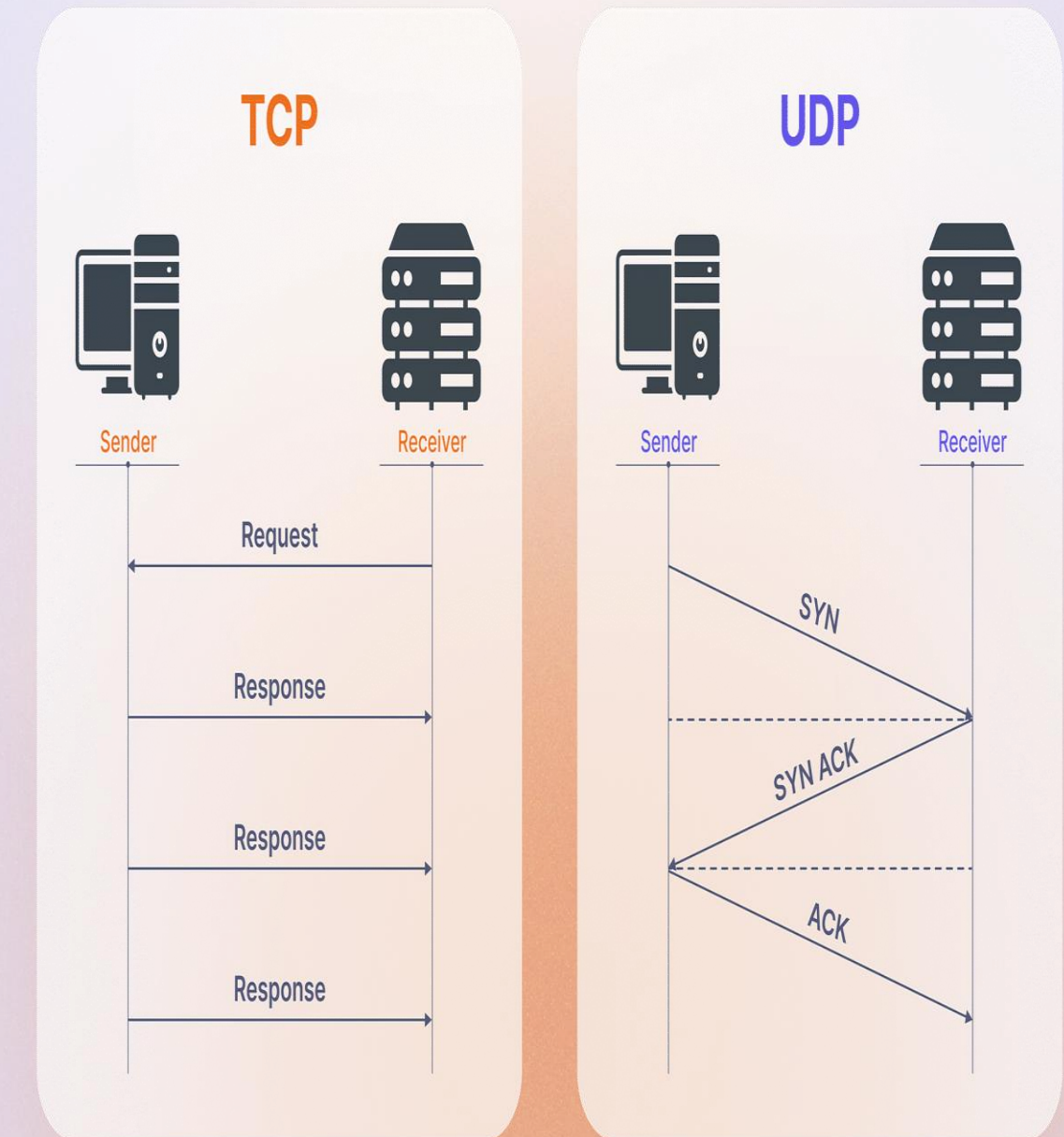
- Reliable, connection-oriented
- Guarantees data delivery
- Error checking and correction
- Slower but more secure
- Used for: Web browsing, email, file transfer

UDP (User Datagram Protocol)

- Fast, connectionless
- No delivery guarantee
- Minimal error checking
- Faster but less reliable
- Used for: Gaming, streaming, DNS

Understanding the differences between TCP and UDP is crucial for designing and troubleshooting network applications, as the choice of protocol directly impacts performance, reliability, and security.

TCP Vs UDP



OSI vs TCP/IP: Side by Side

While both the OSI and TCP/IP models provide a framework for understanding network communication, they differ in their structure and the level of detail they represent. The OSI model is more conceptual, breaking down communication into seven distinct layers, while the TCP/IP model is more practical and aligns closely with the internet's actual architecture.

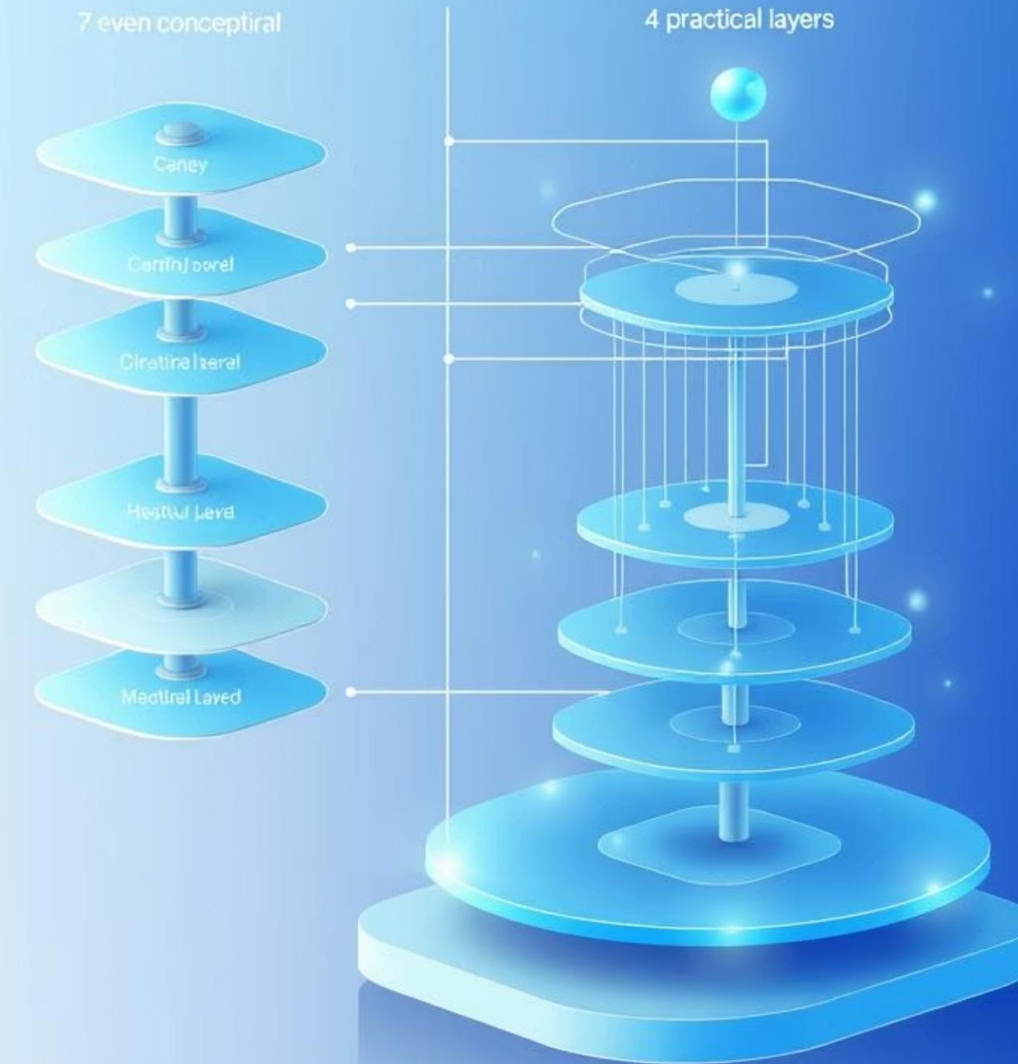
OSI Model (7 Layers)

- Application
- Presentation
- Session
- Transport
- Network
- Data Link
- Physical

TCP/IP Model (4 Layers)

- Application (combines OSI's App, Presentation, Session)
- Transport
- Internet (same as OSI's Network)
- Network Access (combines OSI's Data Link, Physical)

Understanding these differences is crucial for grasping how modern networks function and how various protocols interact within these layered architectures.





Where Did IoT Stack Come From?

Initially, the internet primarily connected computers in a client-server model, facilitating communication and data exchange between larger, more powerful machines.

Over time, mobile phones integrated into this digital landscape, expanding connectivity beyond traditional desktops and laptops. This significant shift broadened the scope of internet usage and device interaction.

As technology advanced, companies began to explore the idea of connecting virtually "everything" – from everyday household appliances like fridges and lights to wearable devices, sensors, and even cars. This ambitious vision gave birth to the concept of the Internet of Things (IoT).

However, connecting these diverse, often tiny and resource-constrained devices presented a unique challenge. Unlike computers, many IoT devices operate with weak processors, limited memory, and finite battery life, making them incompatible with the heavy-duty communication protocols designed for traditional internet infrastructure.

To overcome these limitations, a new approach was needed. The IoT Stack was created, focusing on lightweight protocols and optimized communication layers specifically tailored for small devices and sensors. It emphasizes:

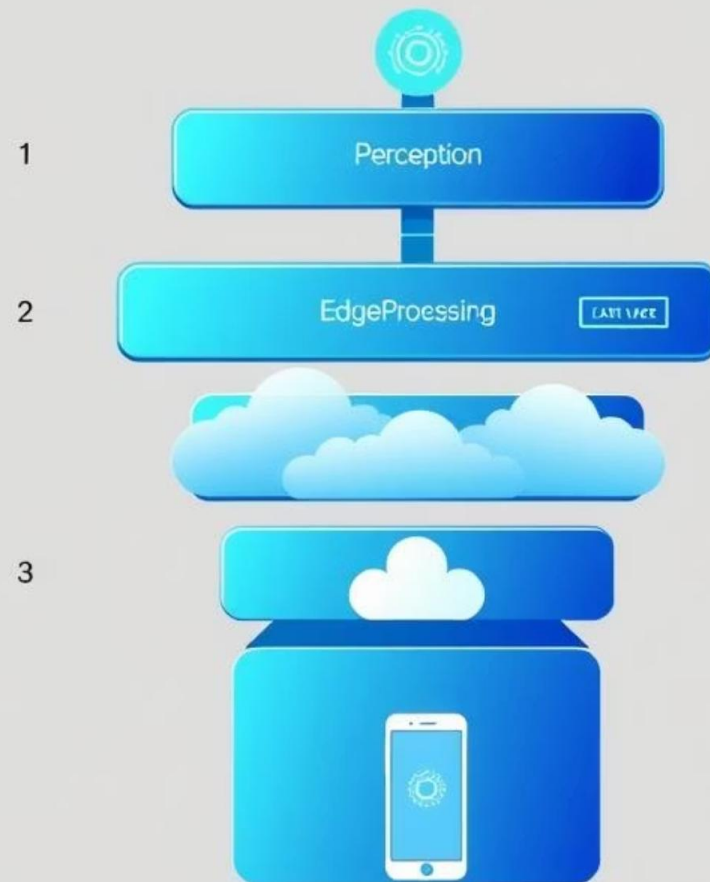
- Device Hardware/Software optimization
- Lightweight Protocols for efficient data transfer
- Cloud Platforms for scalable data storage and processing

Similar in concept to the TCP/IP Stack, the IoT Stack provides a structured framework that enables these constrained devices to communicate effectively and participate fully in the interconnected world of IoT.



IoT Stack (Theoretical Model)

The IoT Stack, in its theoretical model, breaks down the complex architecture of an Internet of Things system into five distinct, interacting layers. This layered approach helps to understand the different functionalities and technologies involved in connecting physical devices to the digital world and delivering meaningful services to users.



Application Layer

What users see - mobile app controlling lights, dashboard showing readings, alerts on phone

Middleware/Platform Layer

Connects devices to cloud, includes Message Brokers like MQTT, platforms like Firebase, AWS IoT, Azure IoT - acts as "translator" between devices and applications

Edge/Processing Layer

Intermediate devices (Gateway or Edge Device) for initial processing, like Raspberry Pi or ESP32 for filtering or compressing data

Network Layer (Communication Layer)

All communication protocols: Wi-Fi, Bluetooth, Zigbee, LoRa, NB-IoT - transfers data from devices to outside world

Perception Layer (Hardware Layer)

Sensors, actuators, cameras - devices that "see/sense" the real world

This structured model helps in designing, implementing, and troubleshooting IoT solutions by clearly defining the responsibilities and interfaces between different components.



IoT Stack (Practical Implementation)



Cloud Applications

Customer interface for dashboards, settings and device control



Cloud Platform

Where data from IoT devices is captured, processed and stored (e.g. Amazon Web Services, Microsoft Azure, Google Cloud)



Communications

How devices connect to the internet and transfer data (Wired, Bluetooth, WiFi, Zigbee, Thread, LTE-M/NB-IoT/2G/3G/4G/5G cellular networks, LoRaWAN)



Device Software

Runs on the device's processor and controls its functionality (Languages used include: Linux, C, Rust, Python, C#)



Device Hardware

Embedded into "Things" in the IoT (ESP32, ARM Cortex, SoCs, gateways, PLCs, interface modules and connectors)



Things

Physical objects found around home, work and everyday life



Device management, data visualisation,
data analytics, alerts and alarms set-up, machine learning



Bare metal server (e.g. SanCloud hosted) or cloud hosted
(e.g. Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP))



Wired, Bluetooth, WiFi, Zigbee, Thread,
LTE-M/NB-IoT/2G/3G/4G/5G cellular networks, LoRaWAN



Languages used include: Linux, C, Perl, Python, Qt



IoT devices include: Sensors, SBCs,
gateways, PLC interface modules and connectors



IoT Stack vs OSI/TCP-IP: Key Differences

OSI/TCP-IP Focus:

- General computer networks and internet
- How data packets move correctly between two devices
- Network communication layers
- Data transmission protocols

IoT Stack Focus:

- Expands the complete picture
- Starts from hardware itself (Sensors) to end user application
- From sensor reading data → to notification on your phone
- Includes device management, cloud platforms, and user interfaces
- Designed for resource-constrained devices
- Lightweight protocols and power efficiency

